

## Laboratorio 05: External Algorithms

---

### Objetivos

---

- Identificar los algoritmos de memoria externa que PostgreSQL utiliza en planes de ejecución reales.
- Implementar en Python External Sorting y External Hashing simulando restricciones severas de RAM.

### Parte 1 (6 pts): Análisis de Planes de Ejecución en PostgreSQL

---

#### Introducción

Las siguientes consultas deben operar sobre atributos sin índice. Con `work_mem` reducida a 64 KB, PostgreSQL no puede materializar los datos en RAM y recurre a algoritmos de memoria externa. El objetivo es identificar cuál algoritmo usa en cada caso.

#### Configuración

Tener cargado en PostgreSQL la base de datos `employees` del Laboratorio 01.

Ejecutar al inicio de cada sesión o transacción:

```
SET LOCAL work_mem = '64kB';
```

#### Consulta 1 (2 pts): ORDER BY sobre hire\_date

```
EXPLAIN ANALYZE  
SELECT * FROM employees.employee ORDER BY hire_date;
```

#### Tareas:

1. Ejecutar el plan con `work_mem = '64kB'` y `work_mem = '2MB'` y registrar el plan.
2. Identificar en cada plan el nodo responsable del ordenamiento y el método utilizado (investigar su funcionamiento).
3. Responder:
  - ¿Qué algoritmo de memoria externa aparece cuando la RAM es insuficiente?
  - ¿Cuántos *merge passes* realizó PostgreSQL? ¿Qué relación tiene esto con el número de runs generados en la Fase 1 del External Sort visto en clase?
  - ¿Cómo cambia el costo al aumentar `work_mem`?

## Consulta 2 (2 pts): GROUP BY sobre atributo from\_date

EXPLAIN ANALYZE

```
SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
```

### Tareas:

1. Ejecutar el plan con `work_mem = '64kB'` y `work_mem = '2MB'` y registrar el plan.
2. Identificar el nodo y estrategia de agregación utilizada.
3. Responder:
  - ¿Qué estrategia usa PostgreSQL para el GROUP BY con poca RAM? ¿Hash Aggregate o Sort + Group Aggregate?
  - Cuando aparece Sort + Group Aggregate, ¿a qué algoritmo de memoria externa corresponde internamente?
  - Cuando aparece Hash Aggregate con desbordamiento a disco (*batches* > 1), ¿a qué algoritmo corresponde?

## Consulta 3 (2 pts): JOIN sobre atributo employee\_id

EXPLAIN ANALYZE

```
SELECT * FROM employees.employee e
JOIN employees.department_employee d ON d.employee_id = e.id;
```

### Tareas:

1. Ejecutar el plan con `work_mem = '64kB'` y `work_mem = '2MB'` y registrar el plan.
2. Identificar el nodo de join y su variante (Hash Join, Merge Join, Nested Loop).
3. Responder:
  - ¿Cuál es el algoritmo de join utilizado con RAM limitada?
  - Si aparece Hash Join con `Batches > 1`, ¿cuántas particiones se generaron? ¿Cómo se relaciona esto con la Fase 1 del External Hashing?
  - Si aparece Merge Join, ¿qué pre-procesamiento tuvo que realizar el motor antes del join?

## Parte 2 (14 pts): Implementación en Python

---

### Introducción

Implementar desde cero los algoritmos External Sorting (Two-Phase Multiway Merge Sort) y External Hashing para GROUP BY, trabajando directamente sobre archivos binarios paginados que simulan un heap file.

### 2.1. (2 pts) Estructura del Heap File

Los datos de `employees.employee` y `employees.department_employee` deben exportarse a archivos binarios con formato de **heap paginado**. Cada archivo se organiza en páginas de tamaño fijo. Los registros dentro de cada página son de longitud fija.

### Tarea:

Implementar por lo menos las siguientes funciones:

```

# Exporta un CSV a un heap file binario paginado.
def export_to_heap(csv_path: str, heap_path: str, record_format: str, page_size: int):
    pass

# Lee una página del heap file y retorna sus registros.
def read_page(heap_path: str, page_id: int, page_size: int) -> list[tuple]:
    pass

# Escribe una lista de registros en la página indicada.
def write_page(heap_path: str, page_id: int, records: list[tuple], record_format: str,
page_size: int):
    pass

# Retorna el número total de páginas del heap file.
def count_pages(heap_path: str, page_size: int) -> int:
    pass

```

## 2.2 (5 pts) External Sorting: Two-Phase Multiway Merge Sort

Implementar el algoritmo TPMMS para ordenar el heap file de `employee` por `hire_date`.

### Restricciones de implementación

- El buffer en RAM está limitado a `BUFFER_SIZE` bytes (configurable: 64 KB, 128 KB, 256 KB).
- El número de páginas que caben en RAM es:  $B = \text{BUFFER\_SIZE} // \text{PAGE\_SIZE}$ .
- En la Fase 2, se usan  $B - 1$  buffers de entrada y 1 buffer de salida.
- No está permitido cargar el archivo completo en memoria en ningún momento.

### Fase 1: Generación de runs

```

"""
Lee B páginas a la vez, las ordena en memoria por el sort_key,
las escribe como archivos temporales de run ordenado.
Retorna la lista de rutas de los runs generados.
"""

def generate_runs(heap_path: str, page_size: int, buffer_size: int, sort_key: str) ->
list[str]:
    pass

```

- El número de runs generados debe ser  $\text{ceil}(\text{total\_pages} / B)$ .

## Fase 2: Multiway Merge

```
"""
    Realiza un k-way merge de los runs usando un min-heap.
    Escribe el resultado ordenado en output_path.
    Usa B-1 buffers de entrada y 1 buffer de salida.
"""
def multiway_merge(run_paths: list[str], output_path: str, page_size: int, buffer_size:
int, sort_key: str):
    pass
```

- Usar `heapq` de Python para el min-heap.
- Cuando el buffer de un run se agota, cargar la siguiente página de ese run desde disco.
- Cuando el buffer de salida se llena, escribirlo a disco y vaciarlo.

### Función principal

```
"""
    Ejecuta TPMMS completo y retorna métricas:
    {
        'runs_generated': int,
        'pages_read': int,
        'pages_written': int,
        'time_phase1_sec': float,
        'time_phase2_sec': float,
        'time_total_sec': float
    }
"""
def external_sort(heap_path: str, output_path: str, page_size: int, buffer_size: int,
sort_key: str) -> dict:
    pass
```

## 2.3. (5 pts) External Hashing: GROUP BY

Implementar External Hashing para resolver:

```
SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
```

### Restricciones de implementación

- El buffer en RAM está limitado a `BUFFER_SIZE` bytes.
- Número de particiones disponibles:  $k = B - 1$  (un buffer de entrada,  $k$  de salida).
- Cada partición debe caber en RAM durante la Fase 2

## Fase 1: Particionamiento

```
"""
Lee el heap file página a página.
Aplica h_p(group_key) % k para asignar cada tupla a una partición.
Escribe las particiones como archivos temporales.
Retorna la lista de rutas de las particiones.
"""
def partition_data(heap_path: str, page_size: int, buffer_size: int, group_key: str) -> list[str]:
    pass
```

## Fase 2: Construcción y agregación

```
"""
Para cada partición:
- Carga en memoria.
- Construye tabla hash con h_r(group_key).
- Acumula COUNT(*) por valor de group_key.
Retorna el diccionario {valor_grupo: count}.
"""
def aggregate_partitions(partition_paths: list[str], page_size: int, buffer_size: int, group_key: str) -> dict:
    pass
```

## Función principal

```
"""
Ejecuta External Hashing completo y retorna:
{
    'result': {valor: count, ...},
    'partitions_created': int,
    'pages_read': int,
    'pages_written': int,
    'time_phase1_sec': float,
    'time_phase2_sec': float,
    'time_total_sec': float
}
"""
def external_hash_group_by(heap_path: str, page_size: int, buffer_size: int, group_key: str) -> dict:
    pass
```

## 2.4 (2 pts) Análisis de rendimiento

Ejecutar ambos algoritmos variando BUFFER\_SIZE CON PAGE\_SIZE = 4096 bytes:

| BUFFER_SIZE | M (páginas en RAM) | Runs / Particiones | Tiempo Fase 1 | Tiempo Fase 2 | Tiempo Total | I/O Total (páginas) |
|-------------|--------------------|--------------------|---------------|---------------|--------------|---------------------|
| 64 KB       |                    |                    |               |               |              |                     |
| 128 KB      |                    |                    |               |               |              |                     |
| 256 KB      |                    |                    |               |               |              |                     |

#### Preguntas de análisis:

1. ¿Cómo varía el número de runs generados en la Fase 1 del TPMMS al duplicar `BUFFER_SIZE` ?
2. ¿Cómo impacta el número de particiones en el costo de I/O del External Hashing?
3. Comparar la solución Python con los tiempos de PostgreSQL de la Parte 1. ¿Qué diferencias observa y a qué las atribuye?
4. ¿En qué escenario preferiría External Sorting sobre External Hashing y viceversa?

## Entregable

---

Un informe en PDF que incluya evidencia del procedimiento (capturas de pantalla) y las respuestas a las preguntas de cada parte del laboratorio. No debe ser muy extenso; se espera un documento ordenado y conciso. Incluya gráficas de tiempo total vs. `BUFFER_SIZE` para ambos algoritmos de la Parte 2. Para verificar la correctitud, compare el resultado del `GROUP BY` en Python con la consulta ejecutada en PostgreSQL.

Además, adjunte el link del repositorio que debe incluir al menos estos archivos:

- `heap_file.py`
- `external_sort.py`
- `external_hashing.py`
- data:
  - `employee.bin`
  - `department_employee.bin`